

# Singleton

## 1. Design Pattern là gì?

Design Pattern là những giải pháp chung cho các vấn đề thường gặp trong thiết kế phần mềm. Chúng không phải là code cụ thể, mà là những khuôn mẫu có thể áp dụng vào nhiều tình huống khác nhau.

### Vai trò chính:

- Giúp code dễ bảo trì và mở rộng.
- Tăng khả năng tái sử dụng.
- Cung cấp ngôn ngữ chung cho các lập trình viên.

### Các nhóm chính:

| Nhóm       | Mục đích                     |
|------------|------------------------------|
| Creational | Tạo đối tượng                |
| Structural | Tổ chức lớp và đối tượng     |
| Behavioral | Giao tiếp giữa các đối tượng |
| Advanced   | Kiến trúc phức tạp           |

➔ Singleton thuộc nhóm Creational

## 2. Singleton Design Pattern

### Định nghĩa

Singleton là một Design Pattern thuộc nhóm Creational, dùng để đảm bảo rằng một lớp chỉ có duy nhất một thể hiện (instance) và cung cấp một điểm truy cập toàn cục (global access point) đến thể hiện đó.

### Ý nghĩa

- Giúp kiểm soát số lượng đối tượng của một lớp (chỉ 1).
- Dùng để quản lý các tài nguyên dùng chung toàn hệ thống.

### Khi nào nên dùng?

- Kết nối cơ sở dữ liệu (Database Connection).
- File log (Logger).
- Quản lý cấu hình hệ thống (Config Manager).

- Quản lý kết nối mạng (Network Manager).

## Các thành phần cốt lõi của Singleton:

Để một class trở thành Singleton, nó bắt buộc phải có 3 thành phần sau:

| Thành phần            | Vai trò                                                                                                                                        |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Static Member         | Lưu trữ thể hiện duy nhất (single instance) của lớp. Vì là static, nó thuộc về class, chỉ tồn tại một bản duy nhất trong toàn bộ chương trình. |
| Private Constructor   | Ngăn chặn việc tạo thể hiện từ bên ngoài (bằng new hoặc khởi tạo trực tiếp). Chỉ cho phép tạo instance bên trong lớp.                          |
| Static Factory Method | Phương thức tĩnh (getInstance()) cung cấp điểm truy cập toàn cục đến instance duy nhất. Đây là cách duy nhất để lấy instance.                  |

## Nguyên lý hoạt động:

- **Lần đầu** gọi getInstance(): static member đang là null → phương thức tạo mới instance bằng cách gọi private constructor → gán vào static member → trả về.
  - **Các lần gọi sau**: static member đã có instance → phương thức trả về instance cũ mà không tạo mới.
- ➔ Đây là cơ chế **Lazy Initialization**: instance chỉ được tạo khi lần đầu tiên được yêu cầu, giúp tiết kiệm tài nguyên.

## 3. Cách triển khai Singleton trong C++

```
class Singleton {
private:
    // 1. Static Member: lưu instance duy nhất
    static Singleton* instance;

    // 2. Private Constructor: không cho tạo từ bên ngoài
    Singleton() {
        cout << "Singleton is Instantiated." << endl;
    }
}
```

public:

// 3. Static Factory Method: điểm truy cập toàn cục

```
static Singleton* getInstance() {
```

```
    if (instance == nullptr) {
```

```
        instance = new Singleton();
```

```
    }
```

```
    return instance;
```

```
}
```

```
void doSomething() {
```

```
    cout << "Something is Done." << endl;
```

```
}
```

```
};
```

// Khởi tạo static member bên ngoài class (bắt buộc trong C++)

```
Singleton* Singleton::instance = nullptr;
```

#### 4. Ưu điểm (Advantages)

- **Kiểm soát instance:** đảm bảo chỉ có một thể hiện duy nhất.
- **Tiết kiệm tài nguyên:** lazy initialization (tạo khi cần).
- **Truy cập toàn cục:** dễ dàng lấy instance từ bất kỳ đâu.
- **Dễ dàng mở rộng:** có thể sửa để hỗ trợ đa luồng hoặc quản lý số lượng instance nhất định.

#### 5. Nhược điểm (Disadvantages)

- **Không an toàn trong đa luồng:** nếu nhiều luồng cùng gọi lần đầu, có thể tạo nhiều instance (race condition).
- **Khó kiểm thử (unit test)** do sử dụng biến toàn cục.

#### 6. Các cách triển khai khác

- **Thread-Safe (Synchronized)** – dùng mutex để an toàn đa luồng.
- **Eager Initialization** – tạo instance ngay khi class được load.
- **Double-Checked Locking** – tối ưu hiệu năng trong đa luồng.

- **Static Inner Class** (Java) – lazy, thread-safe, ngắn gọn.
- **Enum Singleton** (Java) – an toàn nhất, tránh reflection.

**Nguồn :** [Design Patterns](#)  
[Singleton](#)